CAB301 Assignment 1 – Analysis Answer Sheet

Student Name: _____
Student Number: _____

Instructions

Answer all questions clearly and concisely.

For Questions 1–4:

- state a suitable **basic operation**
- count how many times it is performed in the **worst case**
- give the resulting **Big-O time complexity**, using the variables defined below

Any correct worst-case count is acceptable if your reasoning is clear and consistent.

Notation

- `b` = number of books currently stored
- `e` = number of recorded borrow events
- `m` = number of recorded borrow events for a given Dewey number
- `t` = capacity of the `DeweyCountTable`

Question 1 — Starter `ContainsBook`

The starter implementation performs a linear scan over the catalogue.

(a) Basic operation:

(b) Worst-case count:

(c) Big-O complexity:

Question 2 — Completed `SortCatalogue`

For your completed `SortCatalogue` implementation:

(a) Briefly justify your choice of algorithm:

(b) Basic operation:

(c) Worst-case count:

(d) Big-O complexity:

Question 3 — Completed `GetBookIndex`

Assume the catalogue is currently marked sorted.

For your completed `GetBookIndex` implementation:

(a) Briefly justify your choice of algorithm:

(b) Basic operation:

(c) Worst-case count:

(d) Big-O complexity:

Question 4 — Starter `FindBorrowLimitViolator`

Ignore the initial `ContainsBook` check.

The starter implementation:

- scans all `e` events to collect the matching ones
- then repeatedly rescans the `m` matching events to count repeated student IDs

(a) Basic operation:

(b) Worst-case count (in terms of `e` and `m`):

(c) Big-O complexity:

Question 5 — Improved `FindBorrowLimitViolator`

(a) Explain how your method achieves O(e + m log m) or better:

(b) What is its worst-case Big-O complexity in terms of `e` and `m`?

Question 6 — `MostBorrowedSubject` with the auxiliary structure

The completed system maintains a `DeweyCountTable` incrementally.

Assume linear probing takes expected `O(1)` time per table operation.

(a) Time complexity of updating the table during one successful `RecordBorrowEvent`:

(b) Worst-case time complexity of `MostBorrowedSubject()` in terms of `t`:

(c) Why is maintaining the table incrementally more efficient than rescanning all borrow events whenever `MostBorrowedSubject()` is called?

(d) If the system did not maintain `DeweyCountTable` incrementally and instead recomputed counts from all borrow events each time `MostBorrowedSubject()` is called, what would be the worst-case time complexity of one call in terms of e?